

Lista 1 — Introdução & Definição de Programa

Teoria da Computação / Linguagens Formais e Autômatos

Prof. Jefferson O. Andrade

2025/1

1 Introdução

Esta lista de exercícios consiste da resolução de uma série de problemas em Python. O objetivo é praticar programação Python básica e exercitar o conceito de *programa de computador* da forma como formalizada no Capítulo 2 do livro “What can be computed?”, de John MacCormick.

A lista é individual. Você deve resolver os problemas e gear um relatório em $\text{\LaTeX}$ com as suas soluções.¹ O relatório deve conter: (i) o enunciado dos problema; (ii) o código das suas soluções para cada problema; (iii) exemplos de testes da sua solução; e (iv) uma breve explicação da sua solução.

2 Problemas

2.1 Exemplos de Tipos de Problemas

Dê um exemplo de cada um dos seguintes tipos de problemas: (i) tratável, (ii) intratável e (iii) não computável. Descreva cada problema em uma ou duas frases. Não use exemplos já descritos no capítulo 1 do WCBC – faça alguma pesquisa para encontrar exemplos diferentes.

2.2 Intratabilidade e Força Bruta

A Parte II do livro (WCBC) explora a noção de intratabilidade, mas este exercício oferece alguma intuição informal sobre por que certos problemas computacionais podem ser computáveis, mas intratáveis.

1. Suponha que você escreva um programa de computador para descriptografar uma senha criptografada usando “força bruta”. Ou seja, seu programa tenta todas as chaves de criptografia possíveis até encontrar a chave que descriptografa a senha com sucesso. Suponha que seu programa possa testar um bilhão de chaves por segundo. Em média, quanto tempo o programa precisaria se a chave tiver 30 bits de comprimento? E para chaves de 200 bits ou 1000 bits? Compare suas respostas com unidades compreensíveis, como dias, anos, séculos, milênios ou a idade do universo. Em geral, cada vez que adicionamos um único bit ao comprimento da chave, o que acontece com o tempo de execução esperado do seu programa?
2. Considere a seguinte versão simplificada do problema de alinhamento múltiplo de sequências. Recebemos uma lista de 5 sequências genéticas, cada uma com 10 caracteres, e procuramos um alinhamento que insira exatamente 3 espaços em cada uma das sequências. Usamos um programa

¹Se você não tem experiência alguma com $\text{\LaTeX}$, existem vários tutoriais na internet que podem ser úteis. Uma sugestão, é a *playlist* $\text{\LaTeX}$ Tutorial, da professora Jaqueline Silva. Uma outra possibilidade, talvez mais completa e que explora mais recursos – porém em inglês, é a *playlist* $\text{\LaTeX}$ Tutorials, do professor Trefor Bazett.

de computador para encontrar o melhor alinhamento usando força bruta: ou seja, testamos todas as maneiras possíveis de inserir 3 espaços nas sequências de 10 caracteres. Aproximadamente quantas possibilidades precisam ser testadas? Se pudermos testar um bilhão de possibilidades por segundo, qual é o tempo de execução do programa? E se houver 20 sequências genéticas em vez de 5? Se houver N sequências genéticas, qual é o tempo de execução aproximado em termos de N ?

2.3 Implementação de Programas Python SISO

Implemente cada um dos seguintes programas como um programa Python SISO. Em cada caso, você pode assumir que a entrada é válida (ou seja, consiste em uma lista de inteiros formatada corretamente).

1. Escreva um programa que receba como entrada uma lista de inteiros separados por espaços em branco. A saída é uma string representando a soma de todo segundo inteiro na lista. Por exemplo, se a entrada for "58 41 78 3 25 9", então a saída é "53", porque $41 + 3 + 9 = 53$.
2. Escreva um programa, similar ao programa em (1), mas somando todo terceiro elemento da entrada em vez de todo segundo elemento.
3. Escreva um programa de decisão que aceite uma lista de inteiros se a soma de todo terceiro elemento for maior que a soma de todo segundo elemento, e rejeite caso contrário. Seu programa deve importar e usar os programas de (1) e (2).

2.4 Programa de Decisão Python Minimalista

Escreva um programa de decisão em Python que seja o mais curto possível, medido pelo número de caracteres no arquivo de código fonte. Seu programa deve satisfazer a definição formal de um programa Python.

2.5 Reconhecimento de Palíndromos

Escreva uma função em Python chamada `eh_palindromo(texto)` que receba uma string `texto` como entrada e retorne 'yes' se a string for um palíndromo (ou seja, lê-se da mesma forma da esquerda para a direita e da direita para a esquerda, ignorando espaços e diferenças entre maiúsculas e minúsculas), e 'no' caso contrário. Por exemplo, `eh_palindromo("A man a plan a canal Panama")` deve retornar 'yes', e `eh_palindromo("hello")` deve retornar 'no'.

2.6 Contagem de Ocorrências em String

Escreva um programa em Python que receba uma string contendo uma sequência de elementos separados por vírgula e espaço (', '), por exemplo:

```
"maçã, banana, maçã, laranja, banana"
```

O programa deve contar a ocorrência de cada elemento e retornar uma string formatada como:

```
"elemento1:quantidade1, elemento2:quantidade2, ..."
```

No exemplo dado, a saída seria:

```
"maçã:2, banana:2, laranja:1"
```

A ordem dos elementos na string de saída não é importante.

2.7 Inversão de String

Escreva um programa em Python que receba uma string como entrada. O programa deve simular o uso de uma pilha para inverter a ordem dos caracteres da string de entrada e retornar a string invertida como saída. Por exemplo, se a entrada for "python", a saída deve ser "nohtyp".

2.8 Project Euler

Escolha 3 problemas do Project Euler e resolva. Você pode escolher quaisquer problemas, desde que siga as restrições abaixo.

- Ao menos 1 problema com **Difficulty rating** $\geq 10\%$.
- Ao menos 1 problema com **Difficulty rating** $\geq 20\%$.
- Ao menos 1 problema com **Difficulty rating** $\geq 30\%$.

3 Preparação do Relatório

O relatório deve ser preparado em $\LaTeX$. Recomenda-se o uso do compilador Lua $\LaTeX$. Para usuários com pouca experiência em $\LaTeX$ (principalmente) recomenda-se o uso do ambiente Overleaf.²

Uma outra sugestão é utilizar a classe `scrartcl`. A classe `scrartcl` é uma versão mais moderna e flexível da classe `article` (padrão do $\LaTeX$), oferecendo um design tipográfico aprimorado, mais opções de personalização diretamente na declaração da classe e uma melhor integração com outros recursos do pacote KOMA-Script. Ela permite um controle mais fino sobre o layout do seu documento e aproveitar os benefícios de um design tipográfico mais sofisticado.

Além disso, você deve usar o pacote `minted` para exibir os seus códigos fonte no relatório. Para mais informações sobre como usar o pacote `minted`, veja o artigo "Code Highlighting with minted". Uma sugestão é ter todos os arquivos com códigos fonte em um diretório, e.g., `src`, e usar o comando `\inputminted` para exibir os códigos no corpo do seu relatório.

O seu relatório deve conter uma seção para cada problema. As seções de cada problema devem ser subdivididas nas seguintes subseções:

1. **Enunciado:** O enunciado do problema.
2. **Solução:** O código que você escreveu para solucionar o problema.
3. **Descrição:** Uma breve descrição do que (e por quê) o seu código faz.

²Se você já tem experiência com $\LaTeX$ preferir usar outro ambiente, não há problema algum. O uso do Overleaf é meramente uma sugestão por se tratar de um ambiente fácil de usar e por eliminar a necessidade de instalar o ambiente $\TeX$.